

Guía de Resolución: Docker, IntelliJ y PostgreSQL

1. El Problema Principal

Al intentar levantar el entorno con Docker desde IntelliJ y conectar una aplicación Spring Boot, surgieron tres errores encadenados:

1. **Error de Versión de API:** `Status 400: client version 1.24 is too old.`
 - **Causa:** IntelliJ intentaba usar una versión antigua (Legacy) de `docker-compose.exe` que no es compatible con las versiones modernas de Docker Desktop.
 2. **Bucle de Reinicios de la BD:** El contenedor de Postgres se reiniciaba constantemente (`Restarting`).
 - **Causa:** Datos corruptos en el volumen persistente de Windows. Error en logs: `FATAL: could not open directory "pg_notify"`.
 3. **Conexión Rechazada en Java:** `Connection to localhost:5432 refused.`
 - **Causa:** Al estar el contenedor reiniciándose, el puerto 5432 nunca llegaba a abrirse, impidiendo que Java conectara.
-

2. La Solución Paso a Paso

Paso A: Evitar el lanzador de IntelliJ (Error API 1.24)

En lugar de usar el botón "Play" verde de Docker en IntelliJ (que usaba la configuración antigua), pasamos a controlar Docker desde la **Terminal**.

Comando para levantar los servicios ignorando la configuración del IDE:

```
PowerShell
docker compose up -d
```

(El flag `-d` es para "detached mode", lo ejecuta en segundo plano).

Paso B: Arreglar la Corrupción de Datos (Error pg_notify)

El comando `docker compose down -v` no fue suficiente porque el volumen estaba mapeado a una carpeta local de Windows.

1. **Detener Docker:** `docker compose down`
2. **Acción Manual:** Ir a la carpeta del proyecto en el Escritorio y **BORRAR** la carpeta llamada **data**.
 - *Explicación:* Esta carpeta contenía archivos de una instalación fallida anterior. Al borrarla, forzamos a Postgres a regenerarla limpia.
3. **Reiniciar:** `docker compose up -d`
4. **Verificar:** `docker logs postgres_db` (Debe terminar diciendo: *"database system is ready to accept connections"*).

Paso C: Sincronizar Java con Docker

Para que Spring Boot pudiera entrar en la base de datos, el archivo de configuración debía coincidir exactamente con lo definido en el `docker-compose.yml`.

Archivo `docker-compose.yml` (Configuración final):

YAML

```
services:
  db:
    image: postgres:16
    ports:
      - "5432:5432" # Expone el puerto a Windows
    environment:
      POSTGRES_USER: admin
      POSTGRES_PASSWORD: admin123
      POSTGRES_DB: cine
    volumes:
      - ./data:/var/lib/postgresql/data
```

Archivo `src/main/resources/application.properties`:

Properties

```
spring.datasource.url=jdbc:postgresql://localhost:5432/cine
spring.datasource.username=admin
spring.datasource.password=admin123
spring.jpa.database-platform=org.hibernate.dialect.PostgreSQLDialect
spring.jpa.hibernate.ddl-auto=update
```

3. Comandos Útiles para el Futuro

Ver si los contenedores están vivos:

PowerShell
`docker ps`

- *(Si la columna STATUS dice "Up", todo va bien. Si dice "Restarting", hay error).*

Ver por qué falla la base de datos:

PowerShell

`docker logs postgres_db`

-
- **Limpieza total (Si todo falla de nuevo):**
 1. `docker compose down`
 2. Borrar carpeta `./data` manualmente.
 3. `docker compose up -d`